

RESUMO DO MÓDULO

Tratando Erros

Em uma aplicação web, é fundamental implementar um bom tratamento de erros para lidar com exceções e falhas que possam ocorrer durante a execução do código. Isso permite que a aplicação retorne uma resposta amigável para o cliente, informando o que deu errado, além de impedir que o site fique travado ou inacessível.

No Express.js, há várias formas de tratar erros, dependendo do tipo e da origem do erro. Vamos explorar três maneiras de implementar um tratamento de erros eficiente em um site desenvolvido com Express.js.

1. Tratamento de Erros no Middleware de Rota

Uma das maneiras mais comuns de lidar com erros em rotas é usando a estrutura try-catch. Essa abordagem é útil para capturar exceções e enviar respostas específicas quando um erro ocorre dentro de uma rota.

Exemplo:

```
app.get('/users/:id', async (req, res) => {  
  try {  
    const user = await getUserById(req.params.id);  
    res.json(user);  
  } catch (error) {  
    res.status(500).json({ mensagem: 'Erro ao obter usuário' });  
  }  
});
```

Explicação

1. O bloco try é usado para colocar o código que pode gerar uma exceção.
2. Se ocorrer um erro, o controle é transferido para o bloco catch.
3. No catch, uma resposta de erro com o status HTTP 500 é enviada ao cliente.

Essa abordagem é simples e eficaz para capturar erros de rota, mas se o projeto tiver muitas rotas, você acabará repetindo esse padrão em cada uma delas, o que não é ideal para manutenções futuras.

2. Tratamento de Erros no Middleware de Erro Global

Uma maneira mais eficiente de gerenciar erros em todo o aplicativo é usar um middleware de erro global. Esse middleware captura todas as exceções que não foram tratadas diretamente nos middlewares de rota.

Exemplo:

```
app.use((error, req, res, next) => {  
  console.error(error);  
  res.status(500).json({ mensagem: 'Ocorreu um erro inesperado' });  
});
```

Explicação

1. O middleware de erro global é definido com quatro parâmetros: error, req, res e next.
2. Sempre deve ser colocado após todas as rotas e middlewares no aplicativo.
3. Ele captura qualquer erro que não foi tratado nos middlewares anteriores e retorna uma resposta com uma mensagem genérica e um status 500.

Essa abordagem permite centralizar o tratamento de erros e remover a necessidade de try-catch

repetido em cada rota.

3. Tratamento de Erros com express-async-handler

Quando trabalhamos com funções assíncronas (async/await) no Express, a captura de erros se torna um desafio, pois exceções em funções async não são capturadas automaticamente pelos manipuladores de erro padrão.

Para resolver esse problema, podemos usar o pacote express-async-handler. Ele envolve funções assíncronas e lida automaticamente com qualquer exceção lançada, direcionando-a para o middleware de erro global.

Instalando o express-async-handler:

```
npm install express-async-handler
```

Usando o express-async-handler:

```
const express = require('express');  
const app = express();  
const asyncHandler = require('express-async-handler');  
  
// Rota usando express-async-handler  
app.get('/', asyncHandler(async (req, res, next) => {  
  throw new Error('Erro gerado intencionalmente!');  
}));  
  
// Middleware de erro global
```

```
app.use((err, req, res, next) => {  
  console.error(err);  
  res.status(500).send('Ocorreu um erro no servidor!');  
});  
  
app.listen(3000, () => {  
  console.log('Servidor iniciado na porta http://localhost:3000');  
});
```

Explicação

1. express-async-handler é usado para envolver funções assíncronas.
2. Caso ocorra um erro em uma rota assíncrona, ele é automaticamente enviado para o middleware de erro global.

Essa abordagem simplifica o tratamento de erros em rotas assíncronas, eliminando a necessidade de usar try-catch em cada rota.

Conclusão

O tratamento de erros é uma parte essencial de qualquer aplicação web. No Express.js, existem várias maneiras de lidar com erros:

- Tratamento no Middleware de Rota: Usando try-catch para capturar e tratar erros diretamente na rota.
- Middleware de Erro Global: Centraliza o tratamento de erros e garante que todos os erros sejam gerenciados de forma consistente.
- express-async-handler: Simplifica o tratamento de erros em rotas assíncronas.

Implementar um bom tratamento de erros garante que a aplicação continue funcionando de forma

controlada, mesmo em situações inesperadas, proporcionando uma experiência mais estável e feedbacks úteis para o usuário.
